

Search thoughts

- Users can search recipes by **name**, **ingredient**, and optionally **difficulty**.
- Search should allow **partial matching** (example: “panc” matches “pancakes”).
- Output should be a **table** that lists matching recipes.
- Each recipe in results should be **clickable** and go to the detail page.

Data Analysis

- Bar chart: “Most used ingredients”
 - x-axis: ingredient names
 - y-axis: count of recipes containing ingredient
- Pie chart: “Difficulty distribution”
 - labels: Easy/Medium/Intermediate/Hard
 - values: count of recipes per difficulty
- Line chart: “Recipes created over time”
 - x-axis: date
 - y-axis: number of recipes created on that date

Execution flow (you'll add later in step 5)

- Home → Login → Search page → Results table → Click recipe detail → Back → Logout → Success

Installing panda

Creating [forms.py](#)

Adding search and charts in views - [views.py](#)

Adding URL route for search in [urls.py](#) - recipes folder

Creating the Search template

Update the tests

Reflection Questions

1. Favorite app/site data + why analysis helps

A site like CareerFoundry collects data like page clicks, time spent on lessons, completed exercises, and where people get stuck. Analyzing that data helps them improve the course (like spotting lessons everyone struggles with), personalize dashboards, and recommend what to work on next. It also helps them see which features people actually use so they can focus development effort in the right places.

2. QuerySet evaluation: ways to evaluate a QuerySet

A QuerySet is lazy, meaning it doesn't hit the database until you actually need the results. Common ways to force evaluation are: looping over it, converting it to a list, calling `.count()`, `.exists()`, `.first()`, `.last()`, or slicing it (like `[:10]`). Also, methods like `.get()` or `.aggregate()` trigger database work right away because they need an actual result.

3. QuerySet vs DataFrame: pros/cons + why DataFrame helps processing

QuerySets are great for pulling data from the database efficiently and filtering/sorting using Django's ORM. But they aren't built for heavy data manipulation like reshaping data, combining columns, or doing more complex calculations. DataFrames are better for analysis because they're designed for processing data: grouping, aggregations, cleaning, and preparing data for charts is much simpler. The downside is DataFrames usually pull more data into memory, so for big datasets you have to be careful and filter in the QuerySet first.